

Team 6 — Recommended COLMAP Refinement Workflow

Goal

The main goal for the remaining project work should be to build and validate a small COLMAP-based trajectory refinement pipeline.

The intended workflow is:

Phone recording / COLMAP dataset

|



COLMAP refinement

|



Improved COLMAP dataset

|



LichtFeld Studio

|



Improved Gaussian Splat

The important part is **not** to build a new SLAM or Structure-from-Motion system. COLMAP already provides the necessary building blocks. The project contribution should instead be a reliable pipeline around those building blocks.

The final result could eventually behave like a small command-line tool:

```
refine-recording input.zip output.zip
```

where **input.zip** contains the original phone recording / COLMAP data and **output.zip** contains the refined camera trajectory and COLMAP model.

Before automating this, however, the full process should first be tested **manually** in COLMAP and LichtFeld Studio.

1. Important technical point: bundle adjustment needs image correspondences

The original phone recording already provides:

- images,
- camera intrinsics,
- camera poses / trajectory.

However, camera poses alone are not enough for COLMAP's bundle adjustment.

Bundle adjustment improves camera poses by minimizing the reprojection error between:

- 3D scene points, and
- their corresponding 2D observations in multiple images.

Therefore, COLMAP first needs to create the missing visual constraints between the images.

In practice this means:

1. extract image features,
2. match features between images,
3. triangulate 3D points using the existing camera poses,
4. run global bundle adjustment.

Conceptually:

Existing camera poses

+

Image feature correspondences

|



Triangulated sparse 3D model

|



Global bundle adjustment

|



Refined camera poses + refined 3D points

This is the core COLMAP workflow we should investigate.

COLMAP explicitly supports reconstruction from **known camera poses**, so this is a supported use case rather than a workaround.

Useful documentation:

- COLMAP FAQ:
<https://colmap.github.io/faq.html>
- COLMAP CLI documentation:
<https://colmap.github.io/cli.html>
- COLMAP database format:
<https://colmap.github.io/database.html>
- PyCOLMAP API:
<https://colmap.github.io/pycolmap/pycolmap.html>

2. Recommended first experiment

Do **not** start by implementing the complete CLI wrapper.

First prove manually that COLMAP can improve one of our recordings.

The experiment should compare:

Original phone trajectory

|

▼

LichtFeld Studio

|

▼

Gaussian Splat A

against:

Original phone trajectory

|

▼

COLMAP refinement

|



Refined trajectory



LichtFeld Studio



Gaussian Splat B

Everything in LichtFeld should remain the same between A and B.

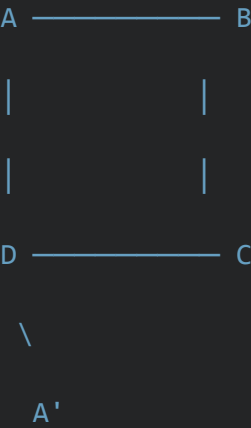
The camera poses should be the main variable.

3. Use a recording where loop drift is clearly visible

The current charging-station recording is useful as a sanity check, but the expected improvement may simply be too small to demonstrate the value of global optimization.

A better test recording will be provided.

Ideally, it should contain a reasonably large loop, for example:



The person physically returns to approximately the same location at the end, but the phone trajectory may have accumulated drift and therefore estimates the final position as **A'**.

For example:

Real world:

start position == end position

Recorded trajectory:

start/end difference = 20–40 cm

If COLMAP finds image correspondences between the beginning and end of the loop, those correspondences create global constraints.

Bundle adjustment can then redistribute accumulated trajectory error and potentially bring the beginning and end of the loop much closer together.

The most important requirement is therefore:

The beginning and end of the recording should contain strong visual overlap.

Good features include:

- textured building façades,
- bricks,
- signs,
- pavement texture,
- poles,
- static street furniture,
- distinctive corners and edges.

Less useful scenes are dominated by:

- grass,
- sky,
- glass,
- repetitive textures,
- moving cars,
- moving people,
- very feature-poor surfaces.

4. Suggested manual COLMAP workflow

Step 1 — Load the existing trajectory

Import/open the existing COLMAP model containing the phone camera poses.

Inspect the trajectory visually.

Capture screenshots before optimization.

If the recording contains a loop, check whether the start and end of the trajectory visibly disagree.

Step 2 — Extract image features

Run COLMAP feature extraction on the original images.

For the first experiment, avoid spending too much time tuning feature parameters.

The default SIFT configuration is a good starting point.

Relevant documentation:

<https://colmap.github.io/features.html>

Step 3 — Match image features

For the first proof-of-concept experiment, **exhaustive matching** is probably the simplest choice.

Why?

Because the datasets are expected to contain only a few hundred images, and exhaustive matching removes uncertainty about whether the correct loop images were considered.

Conceptually:

image 1 ↔ image 2

image 1 ↔ image 3

image 1 ↔ image 4

...

Once the workflow works reliably, we can switch to:

sequential matching

+

loop detection

which is more appropriate for long video-like recordings.

COLMAP's sequential matcher specifically supports loop detection.

Documentation:

<https://colmap.github.io/tutorial.html>

Step 4 — Triangulate points using the known phone poses

This is an important step.

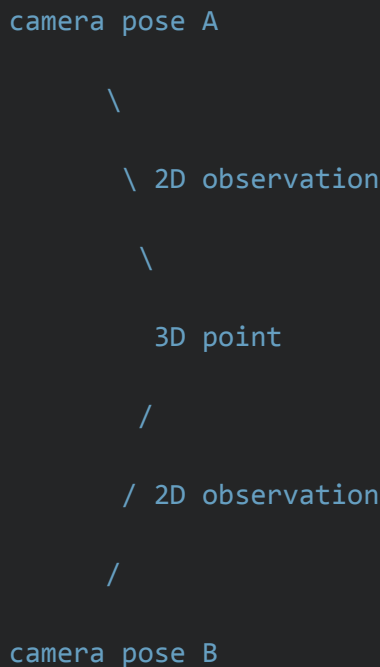
The existing camera poses should be used as the initial registered cameras, while COLMAP triangulates 3D scene points from the feature correspondences.

The relevant COLMAP command is:

```
colmap point_triangulator
```

PyCOLMAP also exposes this directly as triangulation from known camera poses.

Conceptually:



After this step, COLMAP has the visual constraints required for bundle adjustment.

Step 5 — Run global bundle adjustment

Next run:

```
colmap bundle_adjuster
```

Bundle adjustment jointly optimizes:

- camera poses,
- 3D point positions,
- optionally some camera parameters.

The important output for this project is the refined camera trajectory.

If the loop contains good feature correspondences, the global optimization should reduce inconsistency across the trajectory.

Step 6 — Compare the trajectory before and after

Load both reconstructions in COLMAP.

Compare:

Before:

original phone camera poses

with:

After:

triangulation + global bundle adjustment

Capture screenshots from useful viewpoints.

For a loop recording, especially inspect the beginning/end region.

This should become one of the main figures in the final presentation.

Step 7 — Train both versions in LichtFeld Studio

Now create two Gaussian splats.

Version A

Original COLMAP trajectory



LichtFeld Studio



Gaussian Splat A

Version B

Refined COLMAP trajectory



LichtFeld Studio



Gaussian Splat B

Use the same:

- images,
- image resolution,
- training strategy,
- number of iterations,
- LichtFeld version,
- other training parameters.

Then compare the two splats from the same viewpoints.

Useful repository:

<https://github.com/al-cmyk/LichtFeld-Studio>

5. What should be measured?

We do **not** need to build a large custom Gaussian-Splat evaluation framework.

The visual result is the most important output because the actual goal is to improve the generated scene.

However, COLMAP already provides useful numerical information almost for free.

A good evaluation should therefore have three levels.

Level	Question	Suggested evidence
Geometry	Did COLMAP create a more internally consistent reconstruction?	reprojection error / bundle-adjustment cost
Trajectory	Did the camera path actually improve?	before/after trajectory screenshots
Final result	Did the refinement help Gaussian Splatting?	before/after LichtFeld screenshots

6. Reprojection error

COLMAP stores reprojection error for reconstructed 3D points.

This gives us a useful numerical result such as:

Before bundle adjustment:

mean reprojection error = X px

After bundle adjustment:

mean reprojection error = Y px

Ideally:

$Y < X$

This is much more useful than inventing a new arbitrary "quality score."

COLMAP documentation:

<https://colmap.github.io/format.html>

7. Bundle-adjustment optimization statistics

COLMAP / PyCOLMAP also exposes information from the bundle-adjustment solver.

Useful values can include:

- initial optimization cost,
- final optimization cost,
- number of residuals,
- successful optimization steps,
- convergence / termination status.

Eventually the wrapper could report something similar to:

Bundle adjustment

Images:	312
3D points:	84,291
Observations:	436,817
Initial cost:	...
Final cost:	...

Cost reduction: ...

Mean reprojection error:

before: 3.84 px

after: 0.91 px

Solver status:

CONVERGED

This is enough quantitative information for the final presentation.

There is no need to build a complex custom metric framework unless we later discover a specific reason to do so.

8. Gaussian Splat randomness

Gaussian Splat training may contain stochastic components, so two training runs are not necessarily pixel-identical.

However, this should not be a major problem if the trajectory correction is large enough.

If COLMAP changes the camera poses by only a few millimetres, visual differences may indeed be difficult to distinguish from normal training variation.

But if a large loop contains for example:

20–40 cm accumulated drift

then the resulting reconstruction errors can create obvious:

- ghosting,
- duplicated geometry,
- blurry structures,
- inconsistent surfaces.

Correcting a trajectory error of that magnitude should have a much larger visual effect than normal training randomness.

This is another reason why the larger loop recording is useful.

9. Important: do not promise that loop drift becomes exactly zero

The experiment should test the hypothesis that COLMAP can **significantly reduce** global trajectory inconsistency.

It should not assume that:

40 cm drift $\rightarrow$ exactly 0 cm drift

The quality of the correction depends on:

- feature quality,
- feature matching,
- visual overlap,
- scene texture,
- camera calibration,
- viewpoint diversity,
- parallax,
- moving objects,
- optimization convergence.

A better expected result is:

Large loop drift + strong image overlap should allow COLMAP to substantially improve global pose consistency.

10. Coordinate-system issue

There is one implementation detail to remember when the automated wrapper is built.

After bundle adjustment, the optimized reconstruction may no longer be expressed in exactly the same coordinate frame as the original phone model.

For the manual proof of concept this is not necessarily a problem.

For the final automated pipeline, however, we may want:

input coordinate system

$\approx$

output coordinate system

If necessary, the refined reconstruction can be aligned back to the original trajectory using a global similarity transform.

COLMAP also supports pose priors and related alignment functionality, which may be useful later.

For now this should be treated as a **second-stage implementation issue**, not something that blocks the initial experiment.

11. What the eventual wrapper should do

Once the manual experiment works, the complete pipeline could become:

1. Read/unpack input COLMAP recording
2. Import known cameras, intrinsics and poses
3. Create/populate COLMAP database
4. Extract image features
5. Match image features
6. Triangulate points using the existing poses
7. Run global bundle adjustment
8. Optionally retriangulate/refine again
9. Align the optimized reconstruction back to the original coordinate frame
10. Export the refined COLMAP model
11. Create output ZIP
12. Print refinement statistics

From the user's perspective:

```
refine-recording input.zip output.zip
```

The output should be another COLMAP-compatible dataset that can simply be opened in COLMAP or passed manually to LichtFeld Studio.

12. PyCOLMAP vs shell commands

For the first manual experiment, using the COLMAP GUI and CLI is completely fine.

Once the process is understood, **PyCOLMAP** is probably a good way to build the automated black box.

PyCOLMAP exposes many of the underlying COLMAP operations directly from Python, including reconstruction, triangulation and bundle adjustment.

Documentation:

<https://colmap.github.io/pycolmap/pycolmap.html>

A reasonable development strategy is therefore:

Phase 1

COLMAP GUI

↓

Understand the workflow

Phase 2

COLMAP CLI

↓

Verify all required commands

Phase 3

PyCOLMAP / Python wrapper

↓

Automate the workflow

Phase 4

input.zip → output.zip

Do not jump directly to Phase 4.

13. Recommended priority for the remaining work

Highest priority

Get this manual pipeline working:

known poses

↓

feature extraction

↓

feature matching

↓

triangulation

↓

global bundle adjustment

↓

refined trajectory

↓

LichtFeld

↓

visual comparison

Medium priority

Collect simple evaluation information:

- reprojection error,
 - bundle-adjustment cost,
 - trajectory screenshots,
 - Gaussian-Splat screenshots.
-

After that

Automate the successful process into a wrapper.

Lower priority

Only investigate these if there is extra time:

- extensive LichtFeld hyperparameter optimization,
- sophisticated Gaussian-Splat quality metrics,
- alternative research reconstruction systems,
- completely different SLAM/SfM frameworks.

The goal is to make the COLMAP pipeline reliable first.

14. What would count as a successful final result?

A strong final demonstration would show the following.

1. Original phone trajectory

A COLMAP visualization showing visible accumulated drift.

2. Refined trajectory

The same recording after COLMAP triangulation and bundle adjustment, showing improved global consistency.

3. Numerical evidence

For example:

Reprojection error:

before → after

Bundle-adjustment cost:

initial → final

4. Original Gaussian Splat

Rendered from the unrefined phone trajectory.

Highlight visible artifacts such as ghosting.

5. Refined Gaussian Splat

Rendered from the optimized COLMAP trajectory using otherwise identical LichtFeld settings.

6. Automated pipeline

If time allows, demonstrate:

```
refine-recording input.zip output.zip
```

This would tell a very clear project story:

```
The phone provides an approximate camera trajectory.  
COLMAP uses image correspondences to globally refine that trajectory.  
The improved camera poses produce a better Gaussian Splat.
```

Recommended next steps

For the next work session, concentrate on these tasks:

1. Import one of the existing phone recordings into COLMAP.
2. Confirm that the known camera poses load correctly.
3. Extract features from the original images.
4. Run exhaustive feature matching.
5. Run `point_triangulator` using the known poses.
6. Run `bundle_adjuster`.
7. Compare the camera trajectory before and after.
8. Record COLMAP's reprojection / optimization statistics.
9. Create two otherwise identical LichtFeld splats.
10. Compare the resulting scenes visually.
11. Only once this works reliably, begin implementing the automated wrapper.

The most important outcome right now is evidence that the COLMAP refinement actually improves the trajectory and that this improvement is visible in the Gaussian Splat.

Everything else can be built around that result afterward.